

Software Process Activities and Change Management Study Guide

This study guide provides a comprehensive review of the fundamental activities involved in the software engineering process, including specification, development, validation, and evolution. It also examines strategies for managing inevitable system changes and the costs associated with rework.

Part 1: Short-Answer Quiz

Instructions: Answer the following questions in 2-3 sentences based on the provided text.

1. **What is the primary difference between defect testing and debugging?**
 2. **How do user requirements differ from system requirements in a requirements document?**
 3. **What is the purpose of a feasibility study conducted prior to requirements engineering?**
 4. **In the context of software design, what constitutes a "software platform"?**
 5. **Why is requirements engineering considered a particularly critical stage of the software process?**
 6. **What are the three stages of the software testing process?**
 7. **How does the approach to software design documentation differ between agile methods and plan-driven processes?**
 8. **Define the two main approaches used to reduce the costs of rework caused by change.**
 9. **What is the goal of system testing, and how does it differ from component testing?**
 10. **Why is the historical distinction between "development" and "maintenance" becoming increasingly irrelevant?**
-

Part 2: Quiz Answer Key

1. **Defect testing and debugging:** Testing is the process of executing a system with simulated data to establish the existence of defects or bugs. Debugging is a separate process concerned with generating hypotheses about observable anomalies to locate and correct the faults that caused them.

2. **User vs. system requirements:** User requirements are abstract statements of system needs intended for the customer and end-user. System requirements provide a more detailed description of the specific functionality to be provided, intended for the system developers.
 3. **Feasibility study:** A feasibility study is a short-term, inexpensive assessment used to determine if a project is technically and financially realistic. It informs the organization's decision on whether to proceed with a more detailed requirements analysis or a marketing study.
 4. **Software platform:** The software platform is the environment in which the software will execute, consisting of the operating system, database, middleware, and other application systems. Information about this platform is a vital design input because designers must decide how to integrate the new software with these existing systems.
 5. **Criticality of requirements engineering:** This stage is critical because it involves understanding and defining required services and operational constraints. Mistakes made during this stage inevitably lead to more significant problems during later system design and implementation phases.
 6. **Stages of testing:** The three stages are component testing, where individual entities like functions or classes are tested; system testing, where integrated components are checked for interactions and requirement fulfillment; and customer testing, where the system is tested with real data by the user or potential customer.
 7. **Design documentation:** In plan-driven or critical systems, the process yields detailed design documents or diagrams to describe the system precisely. Conversely, in agile methods, design is often interleaved with implementation, and the design may be recorded informally on whiteboards or directly within the program code.
 8. **Rework reduction approaches:** Change anticipation involves activities like prototyping to predict possible changes before high production costs are incurred. Change tolerance involves designing the software and process—often through incremental development—so that changes can be made easily with minimal impact on the rest of the system.
 9. **System vs. component testing:** Component testing focuses on testing individual entities independently by the developers. System testing focuses on the interactions between those components, ensuring the complete system meets functional and non-functional requirements and emergent properties.
 10. **Development and maintenance continuum:** Modern software engineering views development and maintenance as a single evolutionary process. Because very few systems are completely new, software is now seen as a product that is continually changed over its lifetime to respond to evolving requirements and customer needs.
-

Part 3: Essay Format Questions

Instructions: Use the following prompts to develop long-form responses. No answers are provided for this section.

1. **The Interdependence of Design Activities:** Discuss the four main activities of the software design process (architectural, database, interface, and component design). Explain how these activities are interleaved and how new information in one area can necessitate rework in another.
 2. **Agile vs. Plan-Driven Requirements Engineering:** Compare and contrast how requirements are handled in agile methods versus traditional plan-driven processes. Address the timing of specification, the involvement of stakeholders, and the level of formality in the resulting documentation.
 3. **The V-Model and Plan-Driven Validation:** Explain the V-model of development. Describe how test plans for various stages (customer, system, and sub-system) are derived from different development activities and how this structure supports validation in critical systems.
 4. **Software Evolution and the Cost of Change:** Analyze the economic impact of change in large software projects. Discuss why rework is expensive and how the flexibility of software compared to hardware influences the long-term evolution of complex systems.
 5. **The Role of Customer Testing and Beta Testing:** Evaluate the importance of testing software with real customer data versus simulated test data. Discuss how beta testing serves as a bridge between development and general sale for software products.
-

Part 4: Glossary of Key Terms

Term	Definition
Architectural Design	The activity of identifying the overall structure of a system, its principal components (subsystems or modules), their relationships, and their distribution.
Beta Testing	A form of customer testing where a software product is delivered to a number of potential customers who use the system and report problems to the developers.
Change Anticipation	Software process activities, such as prototyping, aimed at predicting potential changes to avoid significant rework later.

Change Tolerance	A design and process approach, often using incremental development, that allows changes to be made to the system with minimal effort and cost.
Component Testing	The stage of testing where individual system components (functions, classes, or groupings) are tested independently by developers.
Debugging	The process of locating and correcting program defects (bugs) by testing hypotheses about observable program behavior.
Feasibility Study	A short, low-cost study to assess if a proposed software system is technically and financially realistic.
Requirements Elicitation	The process of deriving system requirements through observation, discussion with stakeholders, task analysis, and system models.
Requirements Validation	The activity of checking the requirements document for realism, consistency, and completeness to identify errors before implementation.
Rework	The process of redoing work that has already been completed, usually necessitated by changes in requirements or design.
Software Evolution	The continuous process of changing a software system over its lifetime in response to changing requirements and customer needs.
Software Specification	(Also known as requirements engineering) The process of defining the services required from a system and the constraints on its operation.

System Testing	The integration of system components to find errors in interactions and to ensure the system meets its functional and non-functional requirements.
V-model	A plan-driven software process model that relates specific validation activities (testing) to each corresponding stage of the development process.

Source Fidelity Note: *This guide is derived exclusively from the provided excerpts of "ch2.2-Sommerville-.pdf".*